LoomPilot: System Architecture Document

Version 1.0 | 29 October 2026 | Status: Draft for review | Companions: Application Workflow Document, Project Document

1. Purpose and scope

This document describes how LoomPilot (working product name) is built: the components, the technology stack, how the Node backend and the Python AI agents cooperate, the data model, security, and deployment.

LoomPilot is a web platform for a garment and textile manufacturing unit with three user roles:

- Client: submits orders, receives quotations, chats with the AI assistant and tracks order progress.
- Admin: reviews feasibility reports and quotations, approves them, decides on raw material purchases, hands accepted orders to supervisors and handles discount requests.
- Supervisor: submits the daily report (attendance, machine availability, material stock, production) and runs production for assigned orders.

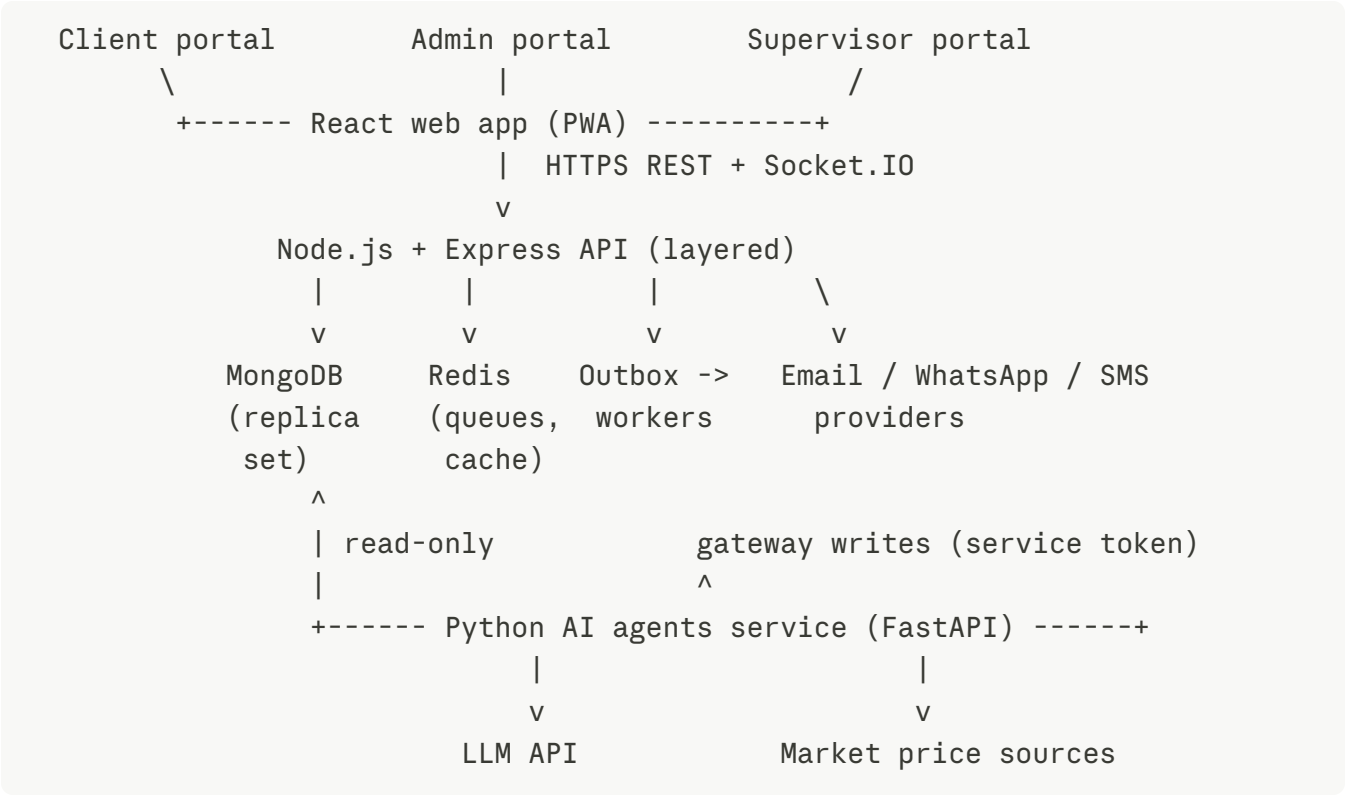
Five Python AI agents analyse feasibility, prepare quotations, watch raw material prices, talk to clients and monitor production. They advise and draft. People approve and decide.

2. Architecture principles

#	Principle	In practice
1	Code computes, the LLM explains	Capacity, ETA, cost, risk buffer, price and variance come from tested Python functions. The LLM writes reasons, summaries and chat replies, and never produces the numbers.
2	Admin approval before the client sees anything	Quotations and every price revision are approved by the admin before they reach the client.

#	Principle	In practice
3	Agents propose, Node writes	Only the Node API writes to MongoDB. Agents read through a read-only user and submit proposals through a gateway.
4	Discounts are human decisions	The chat agent explains and answers. Any discount or extra demand is escalated to the admin.
5	Layered SOLID backend	One folder per layer (controllers, services, repositories, middleware, utils), enforced by dependency rules in CI.
6	Client-safe by design	Clients receive only client-safe fields. Cost, margin and internal reasoning never reach the client API or the chat agent.
7	Auditable and event-driven	Every state change is audited. Notifications and agent triggers go through a transactional outbox.
8	Testable without an LLM	A deterministic fake LLM provider is the default in tests and local development.
9	Tenant-ready	A tenantId on every record and every query, even though v1 serves one manufacturing unit.

3. System context



4. Logical architecture

Tier	Components	Responsibility
Presentation	React web app with three portals (client, admin, supervisor)	Role-based screens, realtime updates, offline entry for the supervisor
API	Express modules by domain, plus the agent gateway	Authentication, role checks, business rules, state machines, transactions, audit
Background	BullMQ workers and schedulers, outbox relay, notifier	Agent triggers, daily market run, retries, notifications, expiry jobs
AI agents	Python FastAPI service in the agents folder	Five agents, domain engine, LLM client, guardrails
Data and external	MongoDB, Redis, file storage, LLM API, market sources, message providers	Storage, queues, files and outbound delivery

The web app is one React application with three role-based portal areas for v1. The areas share one design system and API client, and can be split into separate apps later if the client portal needs its own branding or release cycle.

5. Technology stack

Layer	Technology	Notes
Frontend	React, TypeScript, Vite	PWA via vite-plugin-pwa
UI	Tailwind CSS, shadcn/ui, Recharts	Dashboards, trend charts, progress timelines
Client state and forms	TanStack Query, React Hook Form, Zod, React Router	Zod schemas shared with the API
Realtime (client)	socket.io-client	Alerts, chat, progress updates
i18n	react-i18next	English first, other languages later

Layer	Technology	Notes
Backend	Node.js (current LTS), Express, TypeScript (strict)	Layered, SOLID, constructor injection
Database	MongoDB as a replica set (Atlas in production), Mongoose	Transactions need a replica set
Validation	Zod	Request schemas and OpenAPI generation
Auth	JWT access token, refresh token in an httpOnly cookie, bcrypt	Rotation, reuse detection, login lockout
Queues and jobs	BullMQ on Redis	Retries, backoff, schedules, dead-letter
Realtime (server)	Socket.IO	Authenticated, tenant and role rooms
Documents	pdfmake	Quotation PDFs, customer and internal variants
Email	Nodemailer (SMTP or SES), Mailpit in development	WhatsApp and SMS through provider adapters
API docs	OpenAPI and Swagger UI	Generated from Zod, route coverage tested
AI agents	Python 3.12, FastAPI, Pydantic v2, Uvicorn	In the agents folder
LLM	Anthropic API through the official Python SDK, behind a provider interface	Fake provider is the default in tests
Python data and maths	NumPy, pandas, SciPy	Capacity, volatility and trend calculations
Python data access	PyMongo or Motor (read-only), httpx, tenacity	Retries and timeouts on every call
Python quality	pytest, ruff, mypy, uv, structlog	Type checked and unit tested

Layer	Technology	Notes
Tooling	pnpm workspaces, Docker, GitHub Actions, dependency-cruiser, Vitest, Supertest, mongodb-memory-server, Playwright	One monorepo
Observability	pino, structlog, OpenTelemetry, Sentry	Request ID passed from Node to the agents

6. Repository structure

```
loompilot/
  apps/
    api/src/
      server.ts  app.ts  worker.ts  container.ts   composition root, the only
place that creates concrete classes
      config/        env, logger, db, redis, constants
      routes/        one file per resource
      controllers/   HTTP in, HTTP out, one service call
      services/      business rules, orchestration, state machines
      repositories/  all Mongoose queries, tenant scoped, lean results
      interfaces/    repository, unit of work, agents client, notifier, job
queue, file storage
      models/        Mongoose schemas and indexes
      validators/    Zod request schemas
      mappers/       entity to DTO, role-based and client-safe field hiding
      middleware/    auth, rbac, validate, audit, requestId, idempotency,
serviceAuth, errorHandler
      errors/        AppError and typed errors with stable codes
      utils/         asyncHandler, money (integer paise), dates (tenant
timezone), pagination
      integrations/  agents client, notifiers, pdf, file storage
      jobs/          queues, processors, schedulers
      realtime/      Socket.IO server
      seed/          demo data
    web/src/         portals: client, admin, supervisor; shared api, auth,
i18n, components
  agents/            Python AI agents service, see section 8
  packages/
    shared/          Zod schemas, enums, status tables, error codes, money
```

```
helpers
    agents-client/   TypeScript client generated from the agents OpenAPI spec
  docs/              architecture, workflow, project, ADRs, role matrix,
contracts
  docker-compose.yml
```

7. Node API architecture

7.1 Layers and dependency rules

Dependencies point one way: routes to controllers to services to repositories to models.

Layer	Allowed	Forbidden
Routes	Map paths to controller methods, attach middleware	Logic
Controllers	Read the request, call one service method, map the response	Importing models or repositories, business rules
Services	Business rules, state transitions, audit calls, transactions through the unit of work	Importing express or mongoose, touching the request object
Repositories	Mongoose queries, always tenant scoped, returning plain objects	Business rules
Models	Schemas and indexes	Anything else

Services depend on interfaces injected through constructors. container.ts wires the concrete classes. dependency-cruiser fails the build on any violation.

7.2 Request pipeline

```
request -> requestId -> helmet, CORS, rate limit, body limit -> JWT auth ->
tenant and actor
  -> role check -> Zod validation -> idempotency check -> controller ->
service
  (rules, state machine, audit, outbox event) -> unit of work -> repository
  -> mapper (role and client-safe field hiding) -> response, or errorHandler
(code, message, requestId)
```

7.3 Domain modules

Module	Responsibility
Auth and users	Admin and supervisor users, separate client accounts, sessions, lockout
Clients	Client companies and their portal users
Orders	Client order requests and the full order lifecycle
Feasibility	Stores feasibility reports from the agent, verdicts, reasons, recommendations, admin decisions
Quotations	Drafts, approval, sending, client accept or decline, revisions with reasons, PDF
Market	Watchlist, sources, price observations, price alerts, price history
Purchasing	Admin purchase decisions, supplier orders, goods receipts
Resources	Materials and stock ledger, machines and status logs, workers, products with BOM and routing
Supervisor reports	Daily reports covering attendance, machines, stock and production
Production tracking	Order assignment, progress, monitoring snapshots, phase and percent complete
Chat	Conversations, messages, escalations, admin takeover
Notifications	In-app notifications, preferences, outbox, delivery log
Delivery and payments	Deliveries, invoices and receipts (basic)
Reports	Order pipeline, on-time delivery, price-change impact, quoted versus actual
Platform	Settings, holidays, audit, counters, import, agent run log
Agent gateway	The only write path for agents, see section 8.4

8. AI agents service

8.1 Folder layout

```
agents/
  pyproject.toml  Dockerfile
  app/
    main.py            FastAPI app and the single place concrete clients are
created
    config.py          settings from environment
    api/               routers: feasibility, quotation, market, chat,
monitoring, health
    agents/            base.py (bounded loop: max steps, timeout, token
budget),
                       feasibility_agent.py, quotation_agent.py,
market_agent.py,
                       chat_agent.py, monitoring_agent.py
    tools/             read tools: orders, resources, workforce, prices,
procurement
                       proposals.py: the only write tool, calls the Node
gateway
    domain/            pure functions with no I/O: capacity, feasibility,
cost, risk,
                       timeline, price_trend, variance, recommend
    market/            source adapters (supplier quotes, web search, price
feed), normaliser
    llm/               client.py, providers (anthropic, fake), prompts
(versioned .md files),
                       schemas.py (structured outputs), guardrails.py
    clients/           node_api.py (httpx), mongo_ro.py
    observability/     tracing.py
  tests/               unit, golden (replayed sample orders), api
```

Interfaces are typing.Protocol classes (LLMProvider, NodeGateway, ReadOnlyStore, MarketSource), injected through constructors and FastAPI dependencies.

8.2 The five agents

Agent	Triggered by	Code computes	LLM does	Output through the gateway
Feasibility	New order, admin re-run, after a purchase decision	Daily capacity per process as the lower of labour and machine capacity, minus the load of confirmed orders. Material ready date. ETA. Verdict (feasible, at risk, not feasible). Risk factors. Recovery options: overtime, extra workers, partial shipment, later date, local-market purchase with prices and lead times.	Writes the reasons and the recommendation summary	Feasibility report

Agent	Triggered by	Code computes	LLM does	Output through the gateway
Quotation	Verdict is feasible, or the admin proceeds with an option, or the admin edits and recalculates	Material needs with wastage, purchase quantities, costs, risk buffer from price volatility, margin, price, validity date, ETA basis	Writes the quotation explanation and assumptions	Quotation draft
Market price	Daily schedule, admin refresh	Change versus yesterday and over seven days, volatility, outlier checks, surge flag against thresholds, orders with unpurchased material, repriced estimates	Summarises why prices moved and drafts the client-facing reason	Price observations, price alerts, proposed quotation revisions
Client assistant	Client chat message	Number checks: every figure in a reply must match the stored record	Answers questions, explains price reasons, handles negotiation by explanation, recognises discount or extra-demand requests	Chat replies, escalations to the admin

Agent	Triggered by	Code computes	LLM does	Output through the gateway
Monitoring (recommended)	Supervisor report submitted, daily sweep	Required daily output, actual rate, projected finish, status (on track, at risk, behind), phase and percent complete	Short explanation and recovery suggestions	Monitoring snapshots, alerts, recommendations

8.3 Shared components

- Tools layer: read tools (database, read-only) and one write tool that only creates proposals.
- Domain engine: the pure Python functions. They are the single source of truth for every number, are unit tested against past and sample orders, and Node never re-implements them.
- LLM client and prompts: a provider interface, versioned prompt files, structured outputs validated with Pydantic, token and cost budgets, retries.
- Guardrails: numbers and dates in LLM text are checked against computed values, the chat agent is restricted to client-safe data, step and time limits stop runaway loops.
- Tracing: every run has an agentRunId, request ID, prompt version, model and duration, and is stored in the agent run log visible to the admin.

8.4 Node and agents contract

- Node calls the agents over internal HTTP with a service token: feasibility run, quotation draft and recalculate, market run, chat reply, monitoring run. Each call has a timeout and retries, and on failure the record moves to a failed state with a retry action instead of hanging.
- Agents write back only through gateway endpoints under /internal/v1, using the service token: feasibility-reports, quotation-drafts, quotation-revisions, market-observations, price-alerts, chat-replies, chat-escalations, monitoring-snapshots, alerts.

- Every write carries schemaVersion and agentRunId for idempotency, and the gateway checks that referenced records belong to the same tenant.
- The gateway has no path to stock, orders status, assignments, payments or users. A test allow-lists its routes to prove this.
- Money crosses every boundary as integer paise.
- Contracts (JSON schemas and examples) live in packages/shared, and the agents OpenAPI spec generates the TypeScript client.

8.5 Market data sources

Market sources sit behind a MarketSource interface so they can be swapped. Supplier quotations entered by the admin are the trusted baseline. Web search or price-feed adapters add daily observations. Every observation stores its source, time and a verified flag. A large move from an unverified source is held for admin confirmation before it can trigger revisions.

9. Data architecture

Group	Collections
Identity	users (admin, supervisor), clientAccounts, refreshTokens, auditLogs
Clients and orders	clients, orders, feasibilityReports, quotations, quotationRevisions, orderAssignments, monitoringSnapshots, deliveries, payments
Catalog and resources	products (BOM and routing), materials, stockMovements, reservations, machines, machineStatusLogs, workers, attendance, dailyOutputs
Daily reporting	supervisorDailyReports linking attendance, machine status, stock counts and production
Procurement and market	suppliers, supplierOrders, purchaseDecisions, marketWatchlist, priceObservations, priceAlerts, priceHistory
Conversations	conversations, messages, escalations

Group	Collections
Platform	notifications, outboxMessages, idempotencyKeys, settings, holidays, counters, agentRuns

Data rules:

- Money is stored as integer paise. Dates are strings in the tenant timezone.
- tenantId on every document, and every repository query is scoped by it.
- Stock changes are atomic and recorded in an append-only ledger. Reservations never push available stock below zero.
- Version fields prevent two people overwriting each other. Quotations keep revisions instead of being edited after sending.
- Multi-record changes run in a transaction through the unit of work.

10. Security and access

Capability	Admin	Supervisor	Client
Submit an order	No	No	Yes, own
See orders	All	Assigned orders	Own only, client-safe view
Feasibility report and internal cost breakdown	Yes	No	No
Approve quotation or price revision	Yes	No	No
Accept or decline a quotation	No	No	Yes, own
Assign an order to supervisors	Yes	No	No
Submit the daily report	View only	Yes	No
Chat	All conversations, takeover	No	Own conversation

Capability	Admin	Supervisor	Client
Market prices, purchase decisions	Yes	No	No
Settings, users, audit, agent log	Yes	No	No

- Authentication: short-lived access JWT, hashed refresh tokens with rotation and family revocation on reuse, login lockout, rate limits. Client accounts are a separate login realm linked to a client record and are created by invitation from the admin.
- Client-safe mapping: mappers remove cost, margin and internal fields for client responses. The chat agent can only call client-safe read tools.
- Gateway: constant-time service token check, user tokens rejected, agent writes audited with an agent actor name.
- Data protection: TLS in transit, encryption at rest (database and file storage), secrets only from environment or a secrets manager, log redaction for tokens and passwords, production start refused if default secrets are used.

11. Realtime and background processing

Socket.IO rooms: tenant, role, user, client, and order. Events use one envelope with type, version and payload.

Event	Receiver
order:submitted, feasibility:ready, quotation:pending_approval	Admin
quotation:sent, quotation:revised, order:progress	Client (own)
quotation:accepted, quotation:declined, chat:escalated	Admin
price:alert, revision:proposed	Admin
chat:message	Client and admin on that conversation
report:submitted, order:assigned	Admin, supervisor (assigned)
notification:new	Target user

BullMQ queues: feasibility, quotation, market-daily, chat-reply, monitoring (debounced per order), notifications, maintenance. Schedules run in the tenant timezone: the daily market run, the missing supervisor report check at cutoff, quote expiry, stuck-generation check, and token cleanup.

The transactional outbox stores events and notifications inside the originating transaction. A relay sends them with retries, backoff, channel fallback and a dead-letter state, so nothing is lost if the process restarts.

12. Key runtime flows

1. Order to quotation: client submits, the API stores the order and queues the feasibility run, the agent reads data and posts a report, the verdict is feasible, the quotation agent drafts, the admin approves, the outbox notifies the client.
2. Not feasible: the report carries reasons and options, the admin decides (alternative, purchase, decline). A purchase decision triggers a feasibility re-run.
3. Price surge: the daily run flags a surge, the agent proposes revisions for affected quotations, the admin approves, the client is notified, and the chat agent explains the reason when asked.
4. Chat: a client message is saved, the chat agent replies from client-safe data. A discount request creates an escalation and an admin alert, and the admin can take over the conversation.
5. Handover: the client accepts, the admin is notified, the admin reserves stock and assigns supervisors, and production tracking begins.

13. Deployment and operations

- Local: Docker Compose runs a MongoDB replica set with keyfile authentication, Redis, the API, a worker process (schedulers and queues, kept separate so scaling the API does not duplicate schedulers), the agents service, the web app and Mailpit.
- Production: containers on a managed platform in an Indian region, MongoDB Atlas, managed Redis, S3-compatible storage, a secrets manager, automated backups. The agents service is not publicly reachable and accepts calls only from the API.
- CI: lint, typecheck, dependency rules, unit and integration tests, OpenAPI route coverage and a contract test between the real agents service and the real gateway.
- Observability: structured logs with the request ID, OpenTelemetry traces, Sentry, health and readiness endpoints, and an agent run log with cost per run.

14. Non-functional requirements

Area	Target and approach
Usability	Mobile-first supervisor forms with offline queue, simple client portal
Performance	Quotation and feasibility run asynchronously with live status, target under 60 seconds for a standard order
Reliability	Outbox with retries, idempotent writes, job retries and dead-letter
Security	Roles, tenant scoping, client-safe mapping, encryption, audit
Explainability	Every feasibility report and quotation stores its inputs, assumptions and the agent run details
Scalability	Stateless API and agents, queue-based work, indexed queries
Availability	Backups, health checks, graceful shutdown
Language	English first, i18n ready

15. Testing strategy

- Python domain functions: unit tests with edge cases and golden sample orders.
- Node services: unit tests with in-memory fakes. Routes: integration tests on a MongoDB replica set.
- Security: tenant isolation across all repositories, role by route matrix, client-safe masking, read-only agent database user cannot write, gateway allow-list.
- Contract: the real agents service against the real gateway, and an agent simulator that plays all five agents with fixed payloads.
- End-to-end: order submission through delivery using the simulator.
- Agent quality: replay sample orders and chat transcripts, and check that figures in LLM text match the records.

16. Key decisions, trade-offs and risks

Decision	Reason	Trade-off
Modular monolith with one worker process	Right size for one unit, simple to run	Split services later if load requires it
Python domain engine is the only maths	One source of truth, strong libraries	Node must call the agents service to recalculate
Admin approves every quotation and price revision	Protects margin and client trust	Slower than full automation, which can be relaxed later
Chat agent explains but never grants discounts	Keeps pricing authority with the admin	Clients wait for the admin on concession requests
One web app with three portals	One codebase, shared components	Needs strict route and role separation
Market data behind an adapter	Sources can change without touching agents	Data quality depends on the chosen sources

Risks: unreliable market data (mitigated by supplier quotes as baseline and an unverified flag), thin capacity data in the first weeks (mitigated by conservative buffers and low-data warnings), and price increases after acceptance (needs a price-variation clause in the quotation terms, reviewed by a lawyer).

17. Open items

- Choose the market price sources and whether any paid feed is acceptable.
- Confirm garment routing stages and standard times, and the level of machine detail (machine types versus individual machines).
- Decide the client login method (email and password, or one-time code).
- Confirm hosting provider and region.